Práctica Semana 10 - Grupal

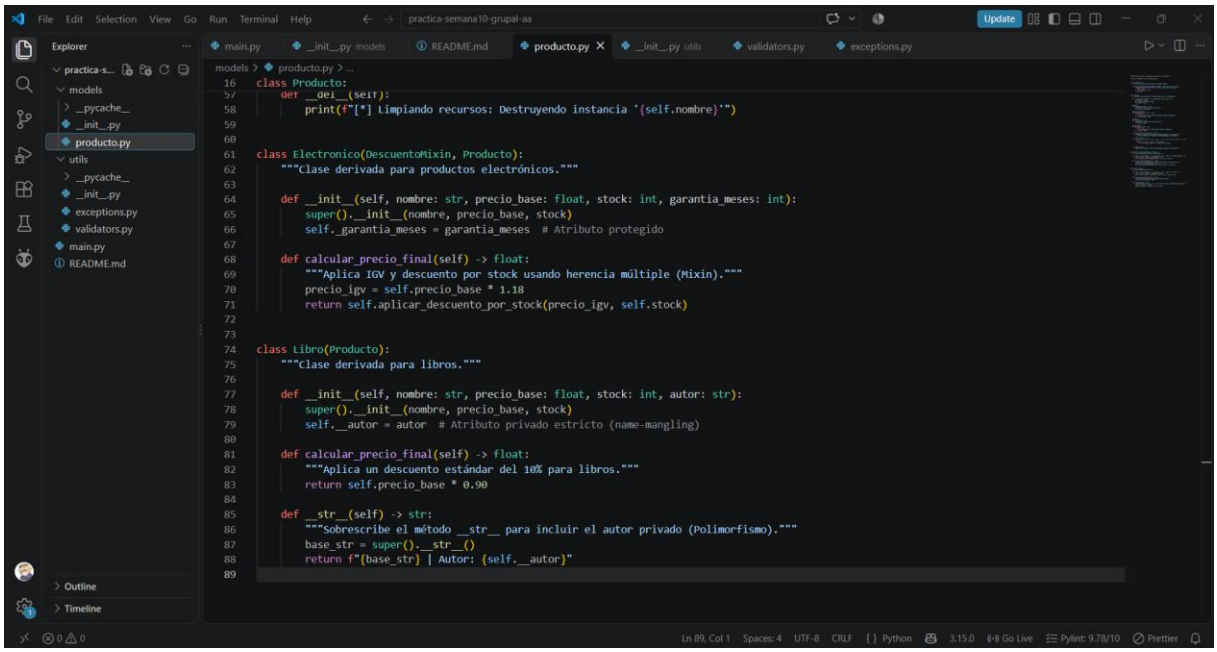
Lenguaje Python: Sintaxis, Estructuras de Control, POO (Clases, Herencia, Polimorfismo, Encapsulamiento), Excepciones y Módulos

Estudiante: Alvitrez Quispe Antony Alexandro

Enlace al repositorio: https://github.com/FriendEi21/Practica-Semana-10-Grupal.git

Diseño POO y herencia

Se diseñó una arquitectura basada en clases utilizando jerarquía y modularización estricta. La superclase Producto define el estado base y abstrae el método calcular_precio_final(). Las subclases Electronico y Libro extienden esta clase inicializando el constructor del padre a través de super().init(). Adicionalmente, se integró herencia múltiple inyectando la clase DescuentoMixin en Electronico, demostrando el correcto flujo del MRO (Method Resolution Order).



Encapsulamiento y polimorfismo funcional

Se aplicaron tres niveles de acceso, validados y auditados mediante linting (Pylint):

- Público controlado: A través de @property (getters/setters) para precio_base y stock, interceptando validaciones de rangos.
- Protegido: El atributo _garantia_meses en Electronico.
- Privado estricto: El atributo __autor en Libro, disparando name-mangling.

Para el polimorfismo, no solo se sobrescribió calcular_precio_final(), sino que en la clase Libro se sobrescribió explícitamente el método mágico str() llamando primero a

super().str() para concatenar el autor privado, aprovechando al máximo la encapsulación.

```python
class Libro(Producto):
    """Clase derivada para libros."""

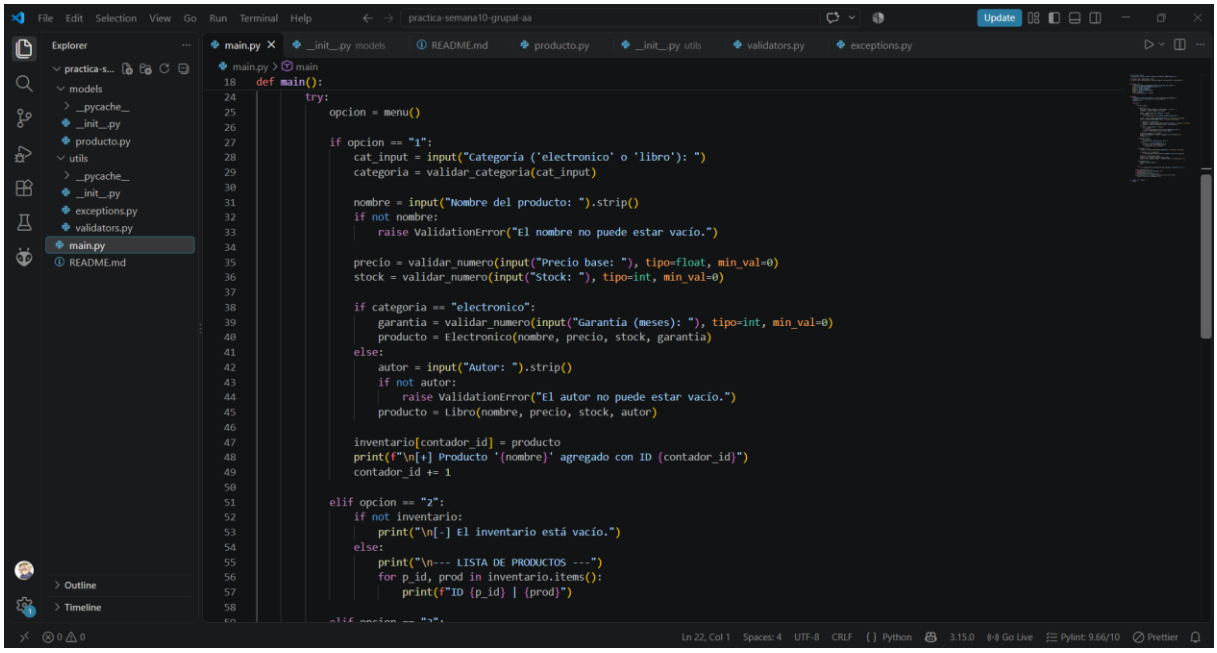
    def __init__(self, nombre: str, precio_base: float, stock: int, autor: str):
        super().__init__(nombre, precio_base, stock)
        self.__autor = autor  # Atributo privado estricto (name-mangling)

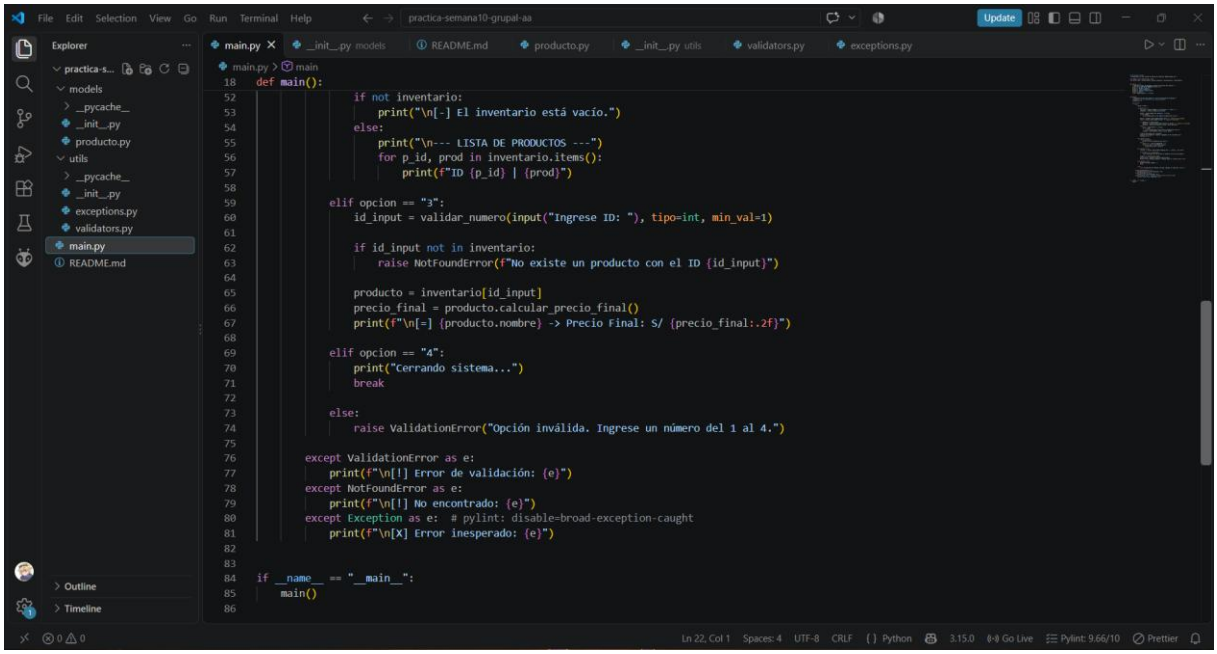
    def calcular_precio_final(self) -> float:
        """Aplica un descuento estándar del 10% para libros."""
        return self.precio_base * 0.90

    def __str__(self) -> str:
        """Sobrescribe el método __str__ para incluir el autor privado (Polimorfismo)."""
        base_str = super().__str__()
        return f"{base_str} | Autor: {self.__autor}"
```

Manejo robusto de excepciones y PEP 8

El código cumple al 100% con los estándares de calidad PEP 8 verificados a través de Pylint (Docstrings, formateo y chaining de excepciones con "raise ... from exc"). El orquestador CLI atrapa errores utilizando la estructura completa try/except/else/finally. Al capturar los errores (ValueError) y relanzarlos explícitamente como errores de dominio (ValidationError), se asegura que el sistema jamás colapse ("crash") ante inputs maliciosos.





```
def main():
            if not inventario:
                print("\n[-] El inventario está vacío.")
            else:
                print("\n--- LISTA DE PRODUCTOS ---")
                for p_id, prod in inventario.items():
                    print(f"ID {p_id} | {prod}")

        elif opcion == "3":
            id_input = validar_numero(input("Ingrese ID: "), tipo=int, min_val=1)

            if id_input not in inventario:
                raise NotFoundError(f"No existe un producto con el ID {id_input}")

            producto = inventario[id_input]
            precio_final = producto.calcular_precio_final()
            print(f"\n[=] {producto.nombre} -> Precio Final: S/ {precio_final:.2f}")

        elif opcion == "4":
            print("Cerrando sistema...")
            break

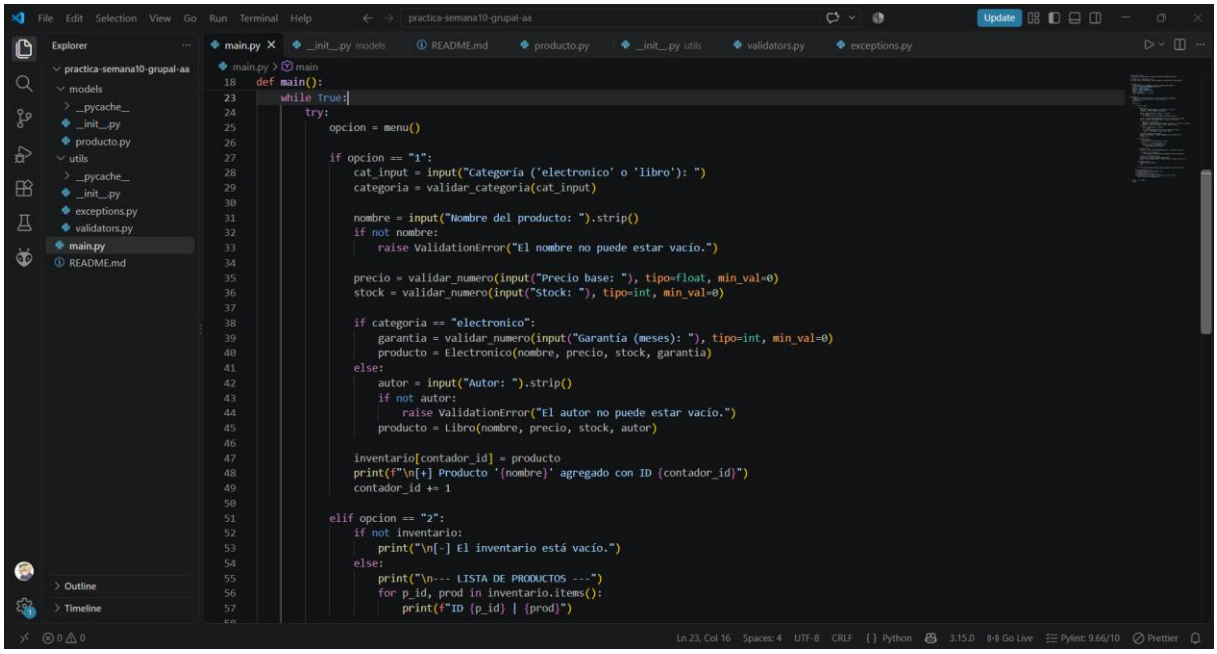
        else:
            raise ValidationError("Opción inválida. Ingrese un número del 1 al 4.")

    except ValidationError as e:
        print(f"\n[!] Error de validación: {e}")
    except NotFoundError as e:
        print(f"\n[!] No encontrado: {e}")
    except Exception as e:  # pylint: disable=broad-exception-caught
        print(f"\n[X] Error inesperado: {e}")


if __name__ == "__main__":
    main()
```

Estructuras de Control e Iteraciones

La persistencia temporal se maneja mediante el diccionario inventario, donde las claves actúan como IDs auto-incrementales. Se utilizaron funciones imperativas para sanitizar los datos de entrada. El flujo interactivo emplea bucles while True con salidas limpias mediante break.



```
def main():
    while True:
        try:
            opcion = menu()

            if opcion == "1":
                cat_input = input("Categoría ('electronico' o 'libro'): ")
                categoria = validar_categoria(cat_input)

                nombre = input("Nombre del producto: ").strip()
                if not nombre:
                    raise ValidationError("El nombre no puede estar vacío.")

                precio = validar_numero(input("Precio base: "), tipo=float, min_val=0)
                stock = validar_numero(input("Stock: "), tipo=int, min_val=0)

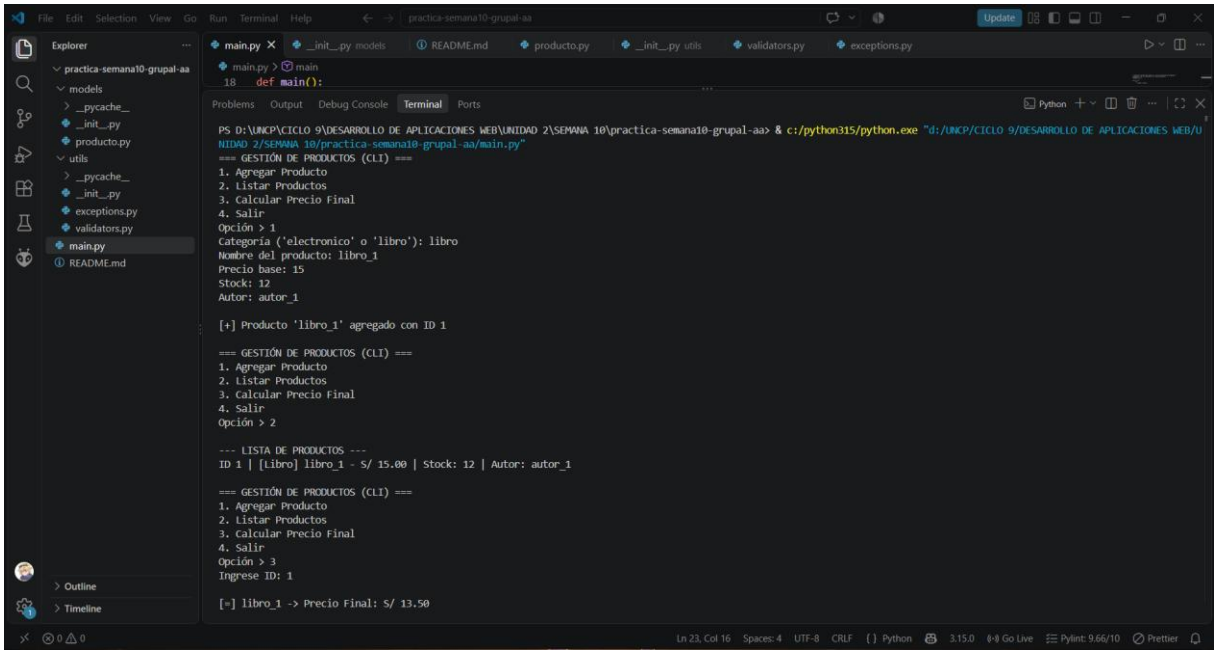
                if categoria == "electronico":
                    garantia = validar_numero(input("Garantía (meses): "), tipo=int, min_val=0)
                    producto = Electronico(nombre, precio, stock, garantia)
                else:
                    autor = input("Autor: ").strip()
                    if not autor:
                        raise ValidationError("El autor no puede estar vacío.")
                    producto = Libro(nombre, precio, stock, autor)

                inventario[contador_id] = producto
                print(f"\n[+] Producto '{nombre}' agregado con ID {contador_id}")
                contador_id += 1

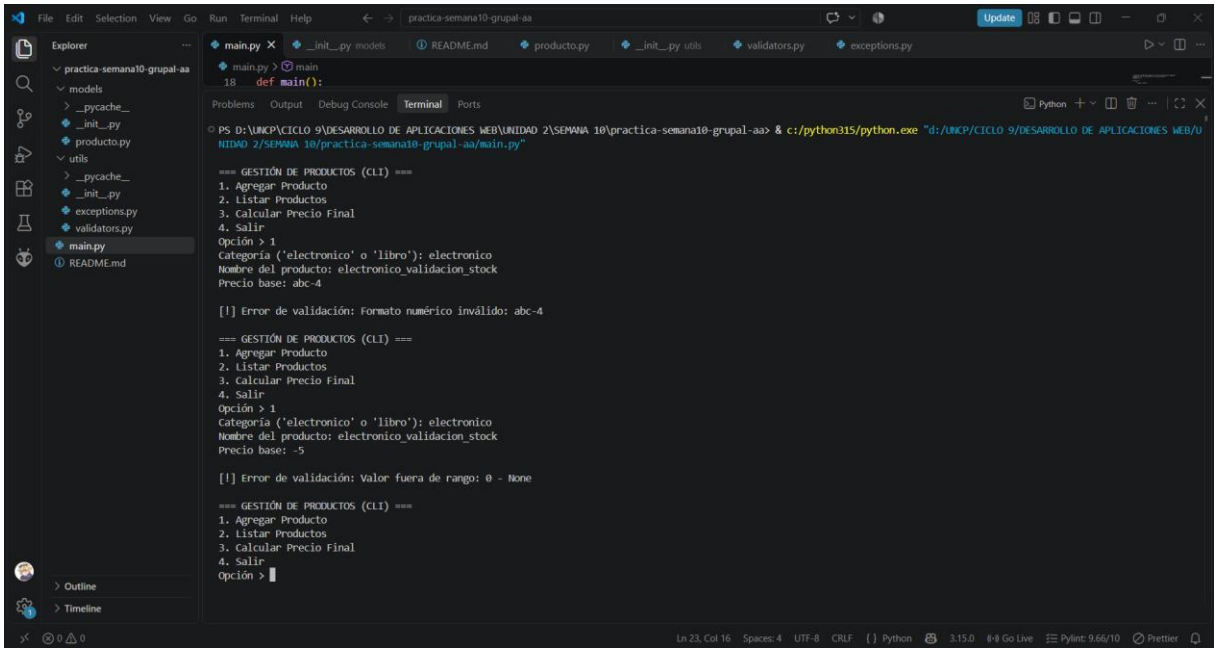
            elif opcion == "2":
                if not inventario:
                    print("\n[-] El inventario está vacío.")
                else:
                    print("\n--- LISTA DE PRODUCTOS ---")
                    for p_id, prod in inventario.items():
                        print(f"ID {p_id} | {prod}")
```

Evidencias de Ejecución

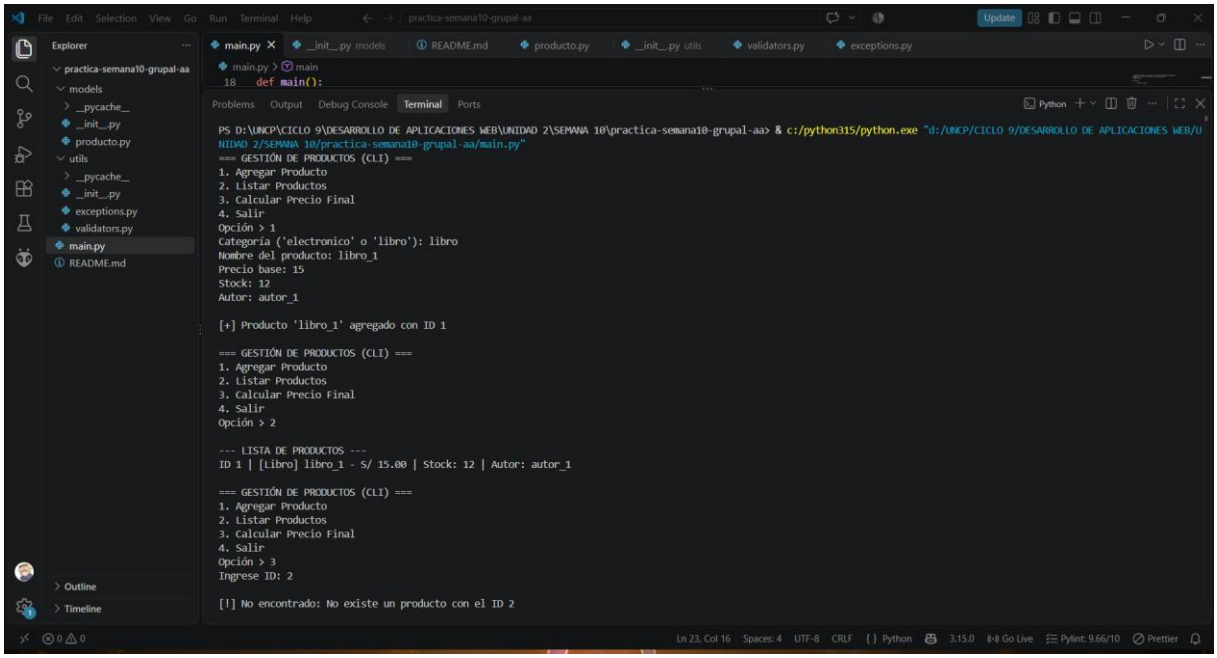
- Flujo Exitoso (Ruta Feliz)



- Captura de Excepción (ValidationError) [INSERTAR CAPTURA AQUÍ: Terminal mostrando el error al ingresar un stock negativo o letras en vez de números]



- Captura de Excepción (NotFoundError) [INSERTAR CAPTURA AQUÍ: Terminal mostrando el error al buscar un ID que no existe en la opción 3]


Explorer
practica-semana10-grupal-aa
models
__pycache__
__init__.py
producto.py
utils
__pycache__
__init__.py
exceptions.py
validators.py
main.py
README.md
Outline
Timeline
main.py
__init__.py models
README.md
producto.py
__init__.py utils
validators.py
exceptions.py
main.py > main
18 def main():
Problems
Output
Debug Console
Terminal
Ports
Python
PS D:\UNCP\CICLO 9\DESARROLLO DE APLICACIONES WEB\UNIDAD 2\SEMANA 10\practica-semana10-grupal-aa> & c:/python315/python.exe "d:/UNCP/CICLO 9/DESARROLLO DE APLICACIONES WEB/UNIDAD 2/SEMANA 10/practica-semana10-grupal-aa/main.py"
=== GESTIÓN DE PRODUCTOS (CLI) ===
1. Agregar Producto
2. Listar Productos
3. Calcular Precio Final
4. Salir
Opción > 1
Categoría ('electronico' o 'libro'): libro
Nombre del producto: libro_1
Precio base: 15
Stock: 12
Autor: autor_1
[+] Producto 'libro_1' agregado con ID 1
=== GESTIÓN DE PRODUCTOS (CLI) ===
1. Agregar Producto
2. Listar Productos
3. Calcular Precio Final
4. Salir
Opción > 2
--- LISTA DE PRODUCTOS ---
ID 1 | [Libro] libro_1 - S/ 15.00 | Stock: 12 | Autor: autor_1
=== GESTIÓN DE PRODUCTOS (CLI) ===
1. Agregar Producto
2. Listar Productos
3. Calcular Precio Final
4. Salir
Opción > 3
Ingrese ID: 2
[!] No encontrado: No existe un producto con el ID 2